# Metody Optymalizacji
# Projekt: PFSP

Jakub Wasilewski 263852

10.04.2026

## Spis treści

**1 Sformułowanie zadania** **2**

**2 Sposób rozwiązywania zadania** **2**

**3 Metody użyte do rozwiązania zadania** **2**

**4 Implementacja** **2**
4.1 Algorytm losowy . . . . . 2
4.2 Algorytm zachłanny . . . . . 2
4.3 Symulowane wyżarzanie (SA) . . . . . 3
4.4 Algorytm ewolucyjny (EA) . . . . . 3
4.5 Kontrola budżetu . . . . . 4

**5 Pliki wejściowe** **4**

**6 Procedura badawcza** **4**

**7 Wyniki badań przed strojeniem** **4**

**8 Strojenie parametrów** **5**
8.1 Rozmiar populacji (`pop_size`) . . . . . 5
8.2 Operatory genetyczne . . . . . 5
8.3 Prawdopodobieństwo krzyżowania (`Px`) . . . . . 6
8.4 Prawdopodobieństwo mutacji (`Pm`) . . . . . 6
8.5 Strojenie parametrów SA . . . . . 7
8.5.1 Wnioski . . . . . 7
8.6 Podsumowanie strojeń EA . . . . . 8

**9 Finalne porównanie algorytmów** **9**
9.1 Wykresy przebiegu EA . . . . . 9

**10 Analiza wyników i wnioski** **10**
10.1 Ranking algorytmów . . . . . 10
10.2 Kluczowe obserwacje . . . . . 11

# 1 Sformułowanie zadania

Celem zadania była implementacja i przebadanie metaheurystyki Algorytmu Ewolucyjnego (EA) dla permutacyjnego problemu przepływowego (PFSP – *Permutation Flow Shop Problem*) oraz porównanie jej z metodami nieewolucyjnymi: algorytmem zachłannym (greedy), symulowanym wyżarzaniem (SA) i losowym przeszukiwaniem.

W PFSP rozpatrywano $n$ zadań i $m$ maszyn. Każde zadanie musiało być wykonane na każdej maszynie w tej samej ustalonej kolejności (maszyna $1 \to 2 \to \ldots \to$ m). Czas przetwarzania zadania $j$ na maszynie $i$ wynosił $p_{ij}$. Poszukiwano permutacji zadań minimalizującej **sumę czasów ukończenia wszystkich zadań na ostatniej maszynie** ($\sum_{j=1}^{J} c(x_j, M)$, tzw. *total completion time*). Nie jest to makespan ($C_{\max}$, czyli czas zakończenia wyłącznie ostatniego zadania), lecz suma wartości $c(x_j, M)$ dla każdego zadania $j$ w kolejności permutacji.

# 2 Sposób rozwiązywania zadania

Rozwiązania były kodowane jako permutacje $n$ zadań. Ocena rozwiązania polegała na obliczeniu sumy czasów ukończenia wszystkich zadań na ostatniej maszynie ($\sum C_j$) poprzez symulację harmonogramu: dla każdej maszyny i każdego zadania (w kolejności permutacji) czas zakończenia wyznaczany był jako maksimum czasu zwolnienia maszyny i czasu zakończenia poprzedniego zadania na tej samej maszynie, powiększony o czas przetwarzania. Na końcu sumowane były czasy ukończenia na ostatniej maszynie dla wszystkich zadań. Wszystkie metaheurystyki operowały bezpośrednio na permutacjach.

# 3 Metody użyte do rozwiązania zadania

- Algorytm losowy (random)
- Algorytm zachłanny (greedy, multi-start)
- Symulowane wyżarzanie (SA)
- Algorytm ewolucyjny (EA)

# 4 Implementacja

## 4.1 Algorytm losowy

- Zastosowano proste losowe przeszukiwanie przestrzeni rozwiązań.
- W każdej iteracji generowana była losowa permutacja $n$ zadań i obliczana jej wartość funkcji celu ($\sum C_j$).
- **Parametry:** liczba iteracji = budżet $B$.
- **Log:** co iterację zapisywany był bieżący koszt; Logger śledził best/avg/worst w czasie.

## 4.2 Algorytm zachłanny

- Zaimplementowano wielostartowy algorytm konstrukcyjny.
- Dla każdego punktu startowego (zadania) budowana była permutacja – w każdym kroku kolejne zadanie wstawiano na pozycję minimalizującą bieżącą wartość $\sum C_j$.
- **Multi-start:** przeprowadzano uruchomienia dla różnych zadań startowych; zwracane było najlepsze znalezione rozwiązanie.

- **Parametry:** liczba restartów (domyślnie = liczba zadań).

### 4.3 Symulowane wyżarzanie (SA)

- Zaimplementowano metaheurystykę inspirowaną procesem wyżarzania metali.
- **Rozwiązanie startowe:** losowa permutacja (nowe w każdym uruchomieniu).
- **Sąsiedztwo:** operator swap – zamiana dwóch losowych pozycji w permutacji.
- **Akceptacja:**
  - Ruch poprawiający ($\Delta < 0$): zawsze był akceptowany.
  - Ruch pogarszający: akceptowany z prawdopodobieństwem $\exp(-\Delta/T)$.
- **Schemat chłodzenia:** geometryczny, $T_{t+1} = \alpha \cdot T_t$, gdzie $\alpha$ to współczynnik chłodzenia.
- **Temperatura początkowa $T_0$:** estymowana była automatycznie ze wzoru $T_0 = -\overline{\Delta^+}/\ln(p_0)$, gdzie $\overline{\Delta^+}$ to średni dodatni delta z 200 losowych ruchów swap, a $p_0 = 0{,}8$ to docelowe prawdopodobieństwo akceptacji ruchu pogarszającego na starcie.
- **Kryterium stopu:** po wykonaniu dokładnie $B$ ewaluacji (kroków pętli).

### 4.4 Algorytm ewolucyjny (EA)

- Zaimplementowano metaheurystykę inspirowaną ewolucją biologiczną.
- **Inicjalizacja:** tworzona była populacja losowych permutacji (rozmiar `pop_size`), każda oceniana – koszt = 1 ewaluacja.
- **Selekcja:** turniejowa – losowano `tour_size` osobników i wybierano najlepszego.
- **Krzyżowanie:**
  - **OX** (Ordered Crossover) – kopiowany był segment z rodzica 1, brakujące elementy uzupełniano w kolejności z rodzica 2, zachowując względną kolejność.
  - **PMX** (Partially Mapped Crossover) – wymieniano segmenty i naprawiano konflikty przez mapowanie pozycji.
  - **CX** (Cycle Crossover) – zachowywane były bezwzględne pozycje genów przez cykl mapowania.
  - Krzyżowanie stosowane było z prawdopodobieństwem `Px`; w przeciwnym razie kopiowany był rodzic 1.
- **Mutacja:**
  - **SWAP** – zamiana dwóch losowych pozycji w permutacji.
  - **INVERSION** – odwrócenie losowego segmentu permutacji.
  - Mutacja stosowana była z prawdopodobieństwem `Pm`.
- **Kryterium stopu:** po wykonaniu dokładnie $B$ ewaluacji.
- **Budżet:** $\texttt{pop_size} \cdot (\texttt{gen} + 1) = B$, gdzie $+1$ uwzględnia koszt inicjalizacji populacji. Stąd $\texttt{gen} = (B - \texttt{pop_size})/\texttt{pop_size}$.

### 4.5 Kontrola budżetu

Wszystkie algorytmy korzystały ze wspólnego licznika wywołań funkcji celu (`eval_counter`) w klasie `PfspInstance`. Po każdym eksperymencie weryfikowany był assert: dla EA – $N_RUNS \cdot B$, dla SA – $N_RUNS \cdot (B + 1)$ (dodatkowe wywołanie z losowego startu).

## 5 Pliki wejściowe

Wykorzystano instancje z zestawu benchmarkowego Taillarda:

- **Łatwe:** tai20_5 (20 zadań, 5 maszyn), tai20_10 (20 zadań, 10 maszyn), tai20_20 (20 zadań, 20 maszyn)
- **Średnie:** tai100_10 (100 zadań, 10 maszyn), tai100_20 (100 zadań, 20 maszyn)
- **Trudne:** tai500_20 (500 zadań, 20 maszyn)

## 6 Procedura badawcza

- Budżet obliczeniowy: $B = 100\,000$ wywołań funkcji celu dla wszystkich konfiguracji.
- Liczba powtórzeń: $N = 5$ niezależnych uruchomień.
- Miara jakości: **średnia** (avg) wyników z $N$ powtórzeń – odporna na pojedyncze szczęśliwe wyniki.
- Parametry bazowe EA: `pop_size=100`, `Px=0.7`, `Pm=0.1`, `tour_size=5`, krzyżowanie OX, mutacja SWAP, inicjalizacja losowa.
- Parametry bazowe SA: $T_0$ auto-estymowane, $\alpha = 0{,}995$.

## 7 Wyniki badań przed strojeniem

Poniższa tabela przedstawia wyniki uzyskane z domyślnymi parametrami ($N = 5$ powtórzeń, $B = 100\,000$). Dla EA zastosowano: `pop=100`, `gen=999`.

Tabela 1: Wyniki bazowe (przed strojeniem)

| Instancja | EA best | EA avg | EA std | SA best | SA avg | SA std |
|---|---|---|---|---|---|---|
| tai20_5 | 14094 | 14190 | 75.7 | 14163 | 14359 | 196.2 |
| tai20_10 | 21432 | 21692 | 257.2 | 21001 | 21547 | 436.8 |
| tai20_20 | 33972 | 34451 | 268.6 | 34320 | 34422 | 60.0 |
| tai100_10 | 314007 | 317724 | 2968.3 | 309957 | 311563 | 1634.6 |
| tai100_20 | 386480 | 389944 | 2594.4 | 385105 | 386373 | 1391.6 |
| tai500_20 | 7218198 | 7238642 | 13201.4 | 6998539 | 7027898 | 17267.2 |

**Wnioski (etap bazowy):**

- Na małych instancjach (tai20_*) EA z domyślną populacją 100 okazał się konkurencyjny lub lepszy od SA.
- Na dużych instancjach (tai100_*, tai500_20) SA wyraźnie dominowało – przewaga rosła ze skalą problemu (dla tai500_20 SA było lepsze o ok. 3%).
- Duże odchylenie standardowe EA wskazywało na niestabilność przy małej populacji.

# 8 Strojenie parametrów

Strojenie przeprowadzono iteracyjnie, każdorazowo utrwalając najlepszy parametr przed przejściem do kolejnego. Budżet $B = 100\,000$ dla wszystkich konfiguracji.

## 8.1 Rozmiar populacji (`pop_size`)

Przetestowano wartości: 100, 200, 500, 1000, 5000, 10000. Liczba generacji dobierana była automatycznie: $\texttt{gen} = (B - \texttt{pop})/\texttt{pop}$.

Tabela 2: Strojenie `pop_size` – avg z 5 uruchomień

| pop_size (gen) | tai20_5 | tai20_10 | tai20_20 | tai100_10 | tai100_20 | tai500_20 |
|---|---|---|---|---|---|---|
| 100 (999) | 14190 | 21692 | 34451 | 317724 | 389944 | **7238642** |
| 200 (499) | 14250 | 21232 | 34118 | 315830 | 389927 | 7263246 |
| 500 (199) | 14119 | 21091 | 33956 | 315084 | 389878 | 7289871 |
| 1000 (99) | 14083 | 20997 | 33947 | 314121 | 390081 | 7298239 |
| 5000 (19) | **14054** | **20975** | **33837** | 331266 | 407276 | 7623762 |
| 10000 (9) | 14296 | 21489 | 34371 | 341050 | 419076 | 7738554 |

**Wnioski:** Ujawniła się odwrotna zależność między optymalnym `pop_size` a trudnością instancji. Na małych instancjach duża populacja (5000) wygrywała – przestrzeń rozwiązań była mała, a różnorodność genetyczna okazała się ważniejsza niż liczba generacji. Na dużych instancjach mała populacja z wieloma generacjami (100, 999 gen) wygrywała – przestrzeń była ogromna, algorytm potrzebował głębokiej iteracyjnej poprawy, a nie szerokiej inicjalizacji. Przy `pop=5000` na tai100/tai500 liczba generacji spadała do 9–19, co praktycznie uniemożliwiało ewolucję.

**Przyjęte wartości:** `pop=5000` dla instancji tai20_*, `pop=500` dla tai100_* i tai500_20.

## 8.2 Operatory genetyczne

Przetestowano macierz 3 operatorów krzyżowania $\times$ 2 operatorów mutacji = 6 kombinacji, przy domyślnych `Px=0.7`, `Pm=0.1`.

Tabela 3: Strojenie operatorów – avg z 5 uruchomień

| Operatory | tai20_5 | tai20_10 | tai20_20 | tai100_10 | tai100_20 | tai500_20 |
|---|---|---|---|---|---|---|
| **OX+SWAP** | **14050** | **20976** | **33804** | **314976** | **388839** | **7269054** |
| OX+INV | 14050 | 20985 | 33839 | 320988 | 394008 | 7396331 |
| PMX+SWAP | 14096 | 21155 | 34012 | 316063 | 390552 | 7295316 |
| PMX+INV | 14114 | 21122 | 34031 | 323640 | 399469 | 7475300 |
| CX+SWAP | 14251 | 21392 | 34253 | 318466 | 392366 | 7373814 |
| CX+INV | 14200 | 21432 | 34289 | 326987 | 405567 | 7536259 |

**Wnioski:** Kombinacja OX+SWAP wygrała we wszystkich sześciu instancjach.

SWAP okazał się lepszy od INVERSION niezależnie od krzyżowania – w PFSP kluczowa jest *względna kolejność* zadań, a nie bezwzględne pozycje. INVERSION odwracał segment, niszcząc starannie wypracowane przez OX bloki kolejności. SWAP zamieniał tylko dwa elementy, zachowując resztę struktury.

CX wypadł najgorzej – jest zaprojektowany do zachowania bezwzględnych pozycji genów, co nie ma znaczenia w PFSP.

PMX uplasował się pośrodku – częściowe mapowanie dawało lepsze wyniki niż CX, ale gorsze niż OX. OX najlepiej zachowywał ciągłe bloki względnej kolejności, co odpowiadało strukturze dobrego harmonogramu w PFSP.

**Przyjęta konfiguracja:** `crossover = OX`, `mutation = SWAP`.

## 8.3 Prawdopodobieństwo krzyżowania (`Px`)

Przetestowano wartości: 0.50, 0.70, 0.90, 0.95, 0.98. Zastosowano operatory OX+SWAP oraz per-instancja `pop_size`.

Tabela 4: Strojenie `Px` – avg z 5 uruchomień

| Px | tai20_5 | tai20_10 | tai20_20 | tai100_10 | tai100_20 | tai500_20 |
|---|---|---|---|---|---|---|
| 0.50 | 14067 | 20977 | 33858 | 318557 | 391250 | 7318585 |
| 0.70 | 14049 | 20978 | **33769** | 316222 | 390330 | 7273954 |
| 0.90 | **14045** | **20947** | 33773 | **311625** | 387485 | 7205858 |
| 0.95 | 14058 | 20964 | 33803 | 313028 | **384112** | 7206759 |
| 0.98 | 14049 | 20963 | 33831 | 311889 | 385569 | **7185858** |

**Wnioski:** Im trudniejsza instancja, tym wyższe `Px` okazywało się korzystniejsze. Na małych instancjach różnice były marginalne (szum przy $N = 5$). Na dużych instancjach wzrost `Px` od 0.50 do 0.98 dał wyraźną poprawę (tai500_20: $-1{,}8\%$).

Krzyżowanie jest operatorem budowania rozwiązań z fragmentów rodziców. W dużej przestrzeni dobre częściowe struktury były rzadkie – krzyżowanie pozwalało je łączyć. Przy niskim `Px` EA degenerował się do lokalnego przeszukiwania z mutacją.

**Przyjęte wartości per-instancja:** tai20_5: 0.90, tai20_10: 0.90, tai20_20: 0.70, tai100_10: 0.90, tai100_20: 0.95, tai500_20: 0.98.

## 8.4 Prawdopodobieństwo mutacji (`Pm`)

Przetestowano wartości: 0.01, 0.05, 0.10, 0.20, 0.40. Zastosowano konfigurację: OX+SWAP, per-instancja `pop_size` i `Px`.

Tabela 5: Strojenie `Pm` – avg z 5 uruchomień

| Pm | tai20_5 | tai20_10 | tai20_20 | tai100_10 | tai100_20 | tai500_20 |
|---|---|---|---|---|---|---|
| 0.01 | **14046** | 20968 | 33812 | 317763 | 388332 | 7218867 |
| 0.05 | 14047 | **20947** | **33782** | 314825 | 387514 | 7207249 |
| 0.10 | 14062 | 20962 | 33783 | 313286 | 385259 | 7189132 |
| 0.20 | 14074 | 20999 | 33818 | 311163 | 383801 | 7180216 |
| 0.40 | 14088 | 21049 | 33862 | **310342** | **383251** | **7168178** |

**Wnioski:** Zaobserwowano odwrotny wzorzec niż przy `Px`: im trudniejsza instancja, tym wyższe `Pm` okazywało się korzystniejsze. Na tai100/tai500 trend był monotonicznie rosnący – `Pm=0.40` wygrało wyraźnie (tai500_20: poprawa o 0.7% vs `Pm=0.01`).

Przy dużej przestrzeni i wysokim `Px` krzyżowanie dominowało reprodukcję – populacja szybko traciła różnorodność i zbiegała do lokalnego minimum. Mutacja przy wysokim `Pm` pełniła rolę dywersyfikatora, regularnie wstrzykując perturbacje. Było to szczególnie istotne gdy przestrzeń była wielowymiarowa i pełna lokalnych minimów. `Px` i `Pm` działały komplementarnie – na trudnych instancjach oba parametry powinny być wysokie.

**Przyjęte wartości per-instancja:** tai20_5: 0.01, tai20_10: 0.05, tai20_20: 0.05, tai100_10: 0.40, tai100_20: 0.40, tai500_20: 0.40.

## 8.5 Strojenie parametrów SA

Przetestowano macierz kombinacji dwóch parametrów:

- **Współczynnik chłodzenia** $\alpha \in \{0.990, 0.995, 0.999\}$
- **Mnożnik temperatury początkowej** $\in \{0.5\times, 1\times, 5\times, 10\times\}$ auto-estymaty ($T_0 = -\overline{\Delta^+}/\ln(0.8)$)

Łącznie 12 kombinacji per instancja, $N = 5$ powtórzeń, $B = 100\,000$.

Tabela 6: Strojenie SA – avg z 5 uruchomień (wybrane wyniki)

| $\alpha$ | t0 mult | tai20_5 | tai20_10 | tai20_20 | tai100_10 | tai100_20 | tai500_20 |
|---|---|---|---|---|---|---|---|
| 0.990 | 0.5× | 14423 | 21812 | 34490 | 312317 | 387086 | **6997211** |
| 0.990 | 1× | 14251 | 21741 | 34415 | 313499 | 385459 | 7027637 |
| 0.990 | 5× | 14316 | 22033 | 34506 | 312030 | 385404 | 7011055 |
| 0.990 | 10× | 14262 | 21790 | 34714 | 312374 | 386978 | 7017959 |
| 0.995 | 0.5× | 14277 | 21609 | 34383 | 313206 | 385331 | 7027496 |
| 0.995 | 1× | 14334 | 21588 | 34611 | 313006 | 385238 | 7024425 |
| 0.995 | 5× | 14290 | 21707 | 34554 | 313482 | 385612 | 7035818 |
| 0.995 | 10× | 14278 | 21700 | 34447 | 313060 | 387038 | 7005802 |
| 0.999 | 0.5× | 14279 | 21585 | 34249 | 313113 | 383319 | 7002763 |
| 0.999 | 1× | 14197 | 21588 | **34213** | **310930** | 385280 | 7019722 |
| 0.999 | 5× | 14205 | 21452 | 34452 | 311430 | **382694** | 7013917 |
| 0.999 | 10× | **14129** | **21291** | 34290 | 311087 | 385608 | 7008148 |

### 8.5.1 Wnioski

**Współczynnik chłodzenia:** $\alpha = 0.999$ wygrał na 5 z 6 instancji. Wolne chłodzenie pozwalało algorytmowi spędzić więcej czasu w fazie eksploracji przy umiarkowanej temperaturze – kluczowe gdy przestrzeń rozwiązań była złożona. Wyjątek stanowiła tai500_20, gdzie $\alpha = 0.990$ okazał się najlepszy, jednak różnice między wszystkimi konfiguracjami na tej instancji wyniosły poniżej 0.2% i mieściły się w szumie statystycznym przy $N = 5$.

**Mnożnik $T_0$:** nie zaobserwowano jednoznacznego trendu – optymalna wartość zależała od instancji. Na małych instancjach (tai20_5, tai20_10) wyższe wartości (10×) dawały lepsze wyniki: przy wolnym chłodzeniu wysoka $T_0$ zapewniała długą fazę eksploracji, a mała przestrzeń pozwalała ją w pełni wykorzystać. Na dużych instancjach (tai100_10, tai100_20) optymalne okazały się wartości 1×–5×, zaś na tai500_20 niska wartość (0.5×) z szybszym chłodzeniem działała najlepiej – zbyt wysoka $T_0$ oznaczała trwonienie budżetu na niemal losowe akceptacje zanim algorytm wszedł w fazę eksploatacji.

**Przyjęte wartości per-instancja:**

| Instancja | $\alpha$ | $T_0$ mult |
|---|---|---|
| tai20_5 | 0.999 | 10× |
| tai20_10 | 0.999 | 10× |
| tai20_20 | 0.999 | 1× |
| tai100_10 | 0.999 | 1× |
| tai100_20 | 0.999 | 5× |
| tai500_20 | 0.990 | 0.5× |

## 8.6 Podsumowanie strojeń EA

Tabela 7: Zestawienie finalnych parametrów per instancja

| | EA | | | | | SA | |
|---|---|---|---|---|---|---|---|
| Instancja | `pop` | `gen` | `Px` | `Pm` | Operatory | $\alpha$ | $T_0$ mult |
| tai20_5 | 5000 | 19 | 0.90 | 0.01 | OX+SWAP | 0.999 | 10× |
| tai20_10 | 5000 | 19 | 0.90 | 0.05 | OX+SWAP | 0.999 | 10× |
| tai20_20 | 5000 | 19 | 0.70 | 0.05 | OX+SWAP | 0.999 | 1× |
| tai100_10 | 500 | 199 | 0.90 | 0.40 | OX+SWAP | 0.999 | 1× |
| tai100_20 | 500 | 199 | 0.95 | 0.40 | OX+SWAP | 0.999 | 5× |
| tai500_20 | 500 | 199 | 0.98 | 0.40 | OX+SWAP | 0.990 | 0.5× |

**Kluczowe obserwacje:**

1. Najważniejszym parametrem okazał się `pop_size` – wpływ na wyniki był największy i jakościowo różny dla małych vs dużych instancji.
2. Operatory OX+SWAP zdominowały wszystkie instancje. INVERSION i CX nie wniosły korzyści dla PFSP.
3. `Px` i `Pm` wykazały lustrzany wzorzec: na trudnych instancjach oba powinny być wysokie. Krzyżowanie eksplorowało przez rekombinację, mutacja utrzymywała różnorodność.
4. SA konsekwentnie wygrywało z EA na dużych instancjach, niezależnie od konfiguracji EA. Różnica na tai500_20 wynosiła ok. 2–3% na korzyść SA.

# 9 Finalne porównanie algorytmów

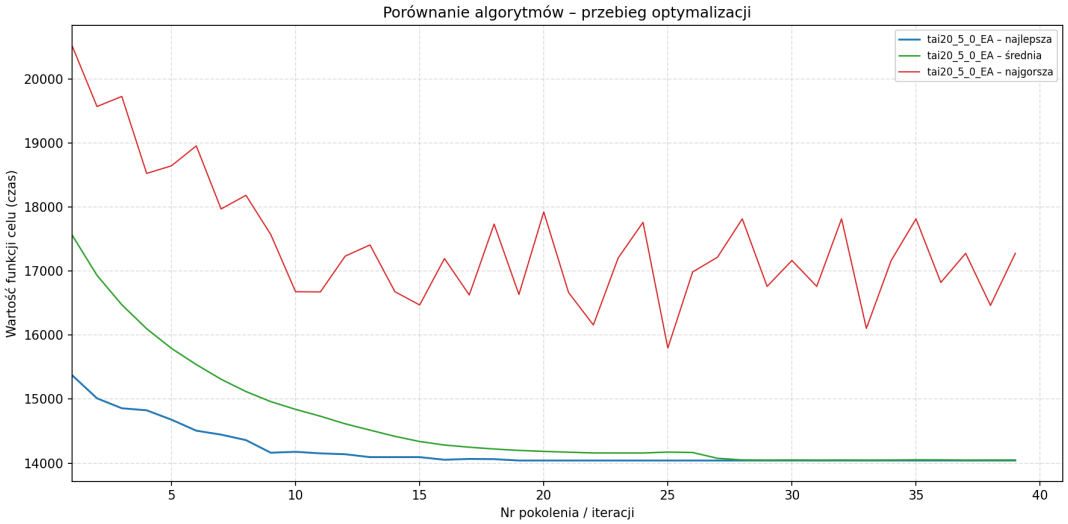
Porównanie przeprowadzono z najlepszymi konfiguracjami znalezionymi w etapie strojenia. Parametry dla każdej metody zestawiono w tabeli w sekcji 8. Zastosowano budżet obliczeniowy $B = 200\,000$ wywołań funkcji celu. Metody niedeterministyczne (Random, EA, SA) uruchomiono $N = 10$ razy niezależnie; wyniki greedy były deterministyczne (jeden przebieg multi-start). Przetestowano wszystkie instancje zestawu Taillarda.

Tabela 8: Finalne porównanie – wyniki z 10 uruchomień ($B = 200\,000$)

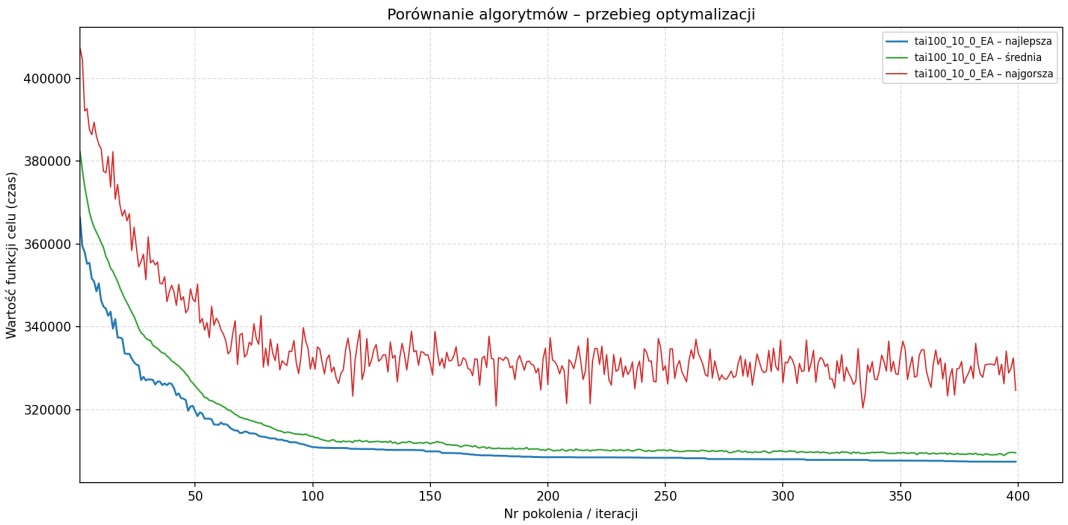
| | Random | | Greedy | EA | | | SA | | |
|---|---|---|---|---|---|---|---|---|---|
| Instancja | best | avg | best | best | avg | std | best | avg | std |
| tai20_5 | 14875 | 15038.3 | 15403 | **14041** | **14042.9** | 5.7 | 14104 | 14261.3 | 102.6 |
| tai20_10 | 21999 | 22563.6 | 24611 | **20911** | **20952.9** | 15.2 | 21186 | 21360.6 | 121.5 |
| tai20_20 | 35323 | 35601.2 | 35103 | **33623** | **33764.0** | 74.3 | 33972 | 34236.9 | 164.3 |
| tai100_10 | 355783 | 357195.5 | 373462 | **307448** | **309690.9** | 1329.3 | 309629 | 311767.5 | 1369.3 |
| tai100_20 | 430692 | 433701.8 | 440486 | **377492** | **382019.0** | 2431.3 | 381329 | 385474.7 | 2729.2 |
| tai500_20 | 7855501 | 7877822.6 | 8407645 | 7062801 | 7093089.4 | 24356.3 | **6942475** | **6977049.1** | 18033.4 |

## 9.1 Wykresy przebiegu EA

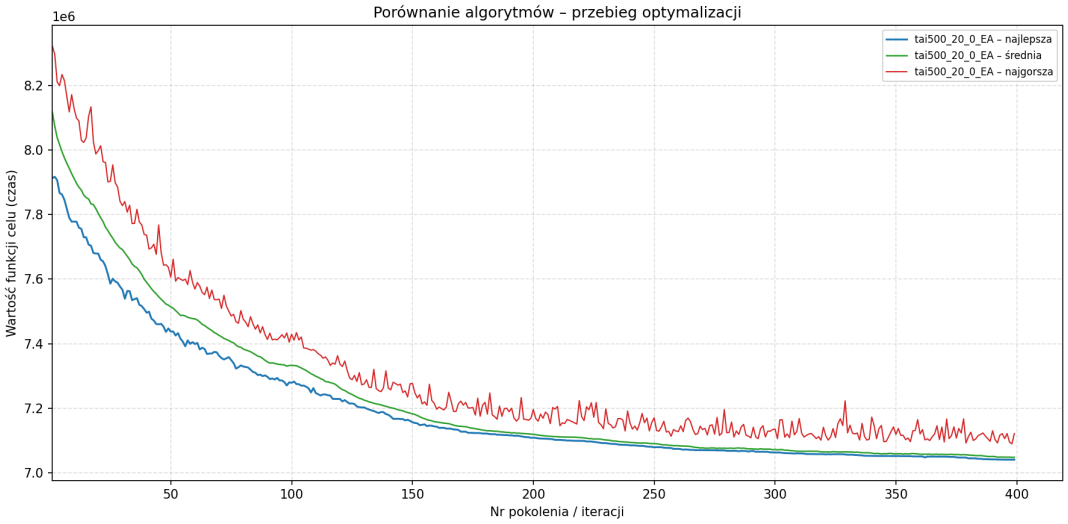
Poniżej przedstawiono przebiegi najlepszego uruchomienia algorytmu ewolucyjnego (best, avg, worst populacji w każdej generacji) dla trzech wybranych instancji.




Rysunek 1: Przebieg EA dla instancji łatwej (tai20_5, pop=5000, gen=39)




Rysunek 2: Przebieg EA dla instancji średniej (tai100_10, pop=500, gen=399)




Rysunek 3: Przebieg EA dla instancji trudnej (tai500_20, pop=500, gen=399)

# 10 Analiza wyników i wnioski

## 10.1 Ranking algorytmów

Na podstawie wyników finalnego porównania ustalono następujący ranking skuteczności:

**Instancje małe (tai20_*):**

1. **EA** – wyraźnie najlepszy, z bardzo niskim odchyleniem standardowym (std=5.7–74.3). Duża populacja (5000) przy małej przestrzeni rozwiązań zapewniła doskonałe pokrycie i stabilność wyników.
2. **SA** – drugi, gorszy od EA o 1–2%.
3. **Random** – trzeci, zbliżony do greedy.
4. **Greedy** – najgorszy na tai20_10 i tai20_20 (24611 vs EA 20911 – różnica 17%!). Greedy padał ofiarą minimów lokalnych już na etapie konstrukcji.

**Instancje średnie (tai100_*):**

1. **EA** – nieznacznie lepszy od SA pod względem avg (tai100_10: 309691 vs 311768, tai100_20: 382019 vs 385475).
2. **SA** – zbliżony do EA.
3. **Random** – znacznie gorszy (tai100_10: avg 357196 vs EA 309691, różnica 15%).
4. **Greedy** – najgorszy (tai100_10: 373462 vs EA 307448, różnica 21%).

**Instancja trudna (tai500_20):**

1. **SA** – najlepszy, avg=6977049, wyraźnie lepszy od EA o ok. 1.6%.
2. **EA** – drugi, avg=7093089.
3. **Random** – znacznie gorszy, avg=7877823 (różnica ok. 12% względem SA).
4. **Greedy** – najgorszy, best=8407645 (gorszy od SA o ponad 20%).

## 10.2 Kluczowe obserwacje

**EA wygrało na małych i średnich instancjach, SA na największej.** Na tai20_* i tai100_* EA po strojeniu okazało się lepsze od SA. Jednak na tai500_20 SA wróciło na prowadzenie – wynik spójny z obserwacjami sprzed strojenia i potwierdzający, że przy bardzo dużej przestrzeni poszukiwań lokalne przeszukiwanie SA skaluje się lepiej niż ewolucja na populacji. Strojenie parametrów EA (zwłaszcza per-instancja pop_size, Px i Pm) miało decydujący wpływ na jakość wyników na pozostałych instancjach.

**Greedy okazało się najgorszą metodą** mimo że jest deterministyczne i szybkie. Ograniczenie budżetowe przy greedy (mała liczba restartów), sprawia, że algorytm ten jest nieskuteczny.

**Random okazało się zaskakująco dobrym** punktem odniesienia – na instancjach tai20 osiągnęło wyniki zbliżone do greedy, a na tai100 było lepsze niż greedy. Wynikało to z bardzo dużej liczby ewaluacji ($B = 200\,000$) – losowe próbkowanie przy takim budżecie pokrywało przestrzeń lepiej niż deterministyczna heurystyka z małą liczbą restartów.

**EA okazało się wyraźnie bardziej stabilne niż SA** – odchylenie standardowe EA było konsekwentnie niższe (np. tai20_5: std=5.7 EA vs 102.6 SA). Duża populacja i mechanizm elitaryzmu (zawsze zachowywane najlepsze rozwiązanie) zapewniały powtarzalność wyników.

**Strojenie per-instancja przyniosło znaczące korzyści.** Porównując wyniki bazowe EA (pop=100, avg tai100_10=317724) z wynikami po strojeniu (pop=500, avg tai100_10=309691) zaobserwowano poprawę o 2.5%. Dla SA poprawa była mniejsza, co sugerowało że SA jest mniej wrażliwe na dobór parametrów przy odpowiednim budżecie obliczeniowym.